

Prólogo

Este libro, "*Programación de Autómatas Siemens con Python en 10 Días*", ha sido desarrollado con el objetivo de proporcionar una guía clara y accesible para quienes desean aprender a programar y comunicarse con **PLCs Siemens** utilizando **Python**. A través de explicaciones concisas y ejemplos prácticos, el lector podrá desarrollar una comprensión sólida de los conceptos fundamentales de la automatización industrial y su integración con el desarrollo de software.

La combinación de **Python y PLCs Siemens** permite una mayor flexibilidad en la supervisión, control y análisis de procesos industriales. Desde la lectura y escritura de datos en un PLC hasta la integración con bases de datos, IoT y sistemas SCADA, este material está diseñado para ser útil tanto para principiantes como para aquellos que buscan expandir sus conocimientos en **automatización, redes industriales e industria 4.0**.

Espero que este libro sea de gran ayuda en tu viaje de aprendizaje y que encuentres en **Python y la automatización industrial** una herramienta poderosa para optimizar procesos, mejorar la eficiencia y llevar tus proyectos a un nuevo nivel.

Licencia Creative Commons

Este libro está licenciado bajo la **Licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional (CC BY-SA 4.0)**. Esto significa que eres libre de:

- ✓ **Compartir** — Copiar y redistribuir el material en cualquier medio o formato.
- ✓ **Adaptar** — Remezclar, transformar y construir sobre el material para cualquier propósito, incluso comercialmente.

Bajo las siguientes condiciones:

- ✓ **Atribución** — Debes dar el crédito adecuado, proporcionar un enlace a la licencia e indicar si se han realizado cambios. Puedes hacerlo de cualquier manera razonable, pero no de una forma que sugiera que el licenciante te respalda a ti o a tu uso.
- ✓ **CompartirIgual** — Si remezclas, transformas o creas a partir del material, debes distribuir tus contribuciones bajo la misma licencia que el original.

Para más información, visita: <https://creativecommons.org/licenses/by-sa/4.0/>

Autor: Oscar de la Cuesta Campillo
@oscardelacuesta



Índice: Programación de Autómatas Siemens con Python en 10 Días

Día 1: Introducción a los Autómatas Siemens y Python

- ¿Qué es un autómata programable (PLC)?
- Tipos de PLC Siemens: S7-200, S7-300, S7-1200, S7-1500
- Beneficios de usar Python para comunicación con PLCs
- Instalación del entorno de desarrollo: Python y bibliotecas necesarias
- Primeros pasos con `snap7`: Instalación y configuración

Día 2: Comunicación con un PLC Siemens usando Python

- ¿Qué es el protocolo S7?
- Conectando Python con un PLC Siemens usando `snap7`
- Configuración de un PLC Siemens para permitir conexiones
- Lectura de datos de un PLC con Python
- Escritura de datos en un PLC desde Python

Día 3: Lectura y Escritura de Variables en el PLC

- Direccionamiento en la memoria del PLC (DB, entradas, salidas, marcas)
- Leyendo entradas digitales
- Escribiendo salidas digitales
- Lectura y escritura de valores en áreas de memoria (DB)
- Manejo de errores en la comunicación

Día 4: Manejo Avanzado de Datos en Python y PLC

- Tipos de datos en los PLC Siemens (BOOL, INT, REAL, STRING)
- Conversión de datos entre Python y PLC
- Uso de estructuras y listas para gestionar datos en Python
- Comunicación en tiempo real con el PLC
- Depuración de errores en la comunicación

Día 5: Monitoreo y Control en Tiempo Real

- Implementación de un sistema de monitoreo en Python
- Control de salidas en tiempo real
- Lectura de sensores y actualización en Python
- Registro de eventos en un log
- Creación de una interfaz gráfica básica con Tkinter



Día 6: Programación de PLCs con TIA Portal y Python

- Introducción a TIA Portal
- Creación de un programa básico en TIA Portal
- Exportación e importación de datos de TIA Portal a Python
- Simulación de un PLC con PLCSIM y conexión con Python
- Pruebas de integración

Día 7: Conectividad Avanzada y Redes Industriales

- Comunicación con múltiples PLCs desde Python
- Uso de MQTT para integración con IoT
- Conexión con bases de datos (MySQL, SQLite) para almacenamiento de datos
- Integración con Node-RED y visualización en dashboards
- Seguridad en la comunicación con PLCs

Día 8: Automatización y Control Remoto

- Implementación de un sistema SCADA en Python
- Control remoto de un PLC desde una aplicación web
- Uso de Flask para crear un servidor de control
- Envío de comandos remotos al PLC
- Monitorización y alarmas en tiempo real

Día 9: Optimización y Mantenimiento

- Estrategias de optimización en la comunicación
- Diagnóstico de fallos en el PLC desde Python
- Técnicas de mantenimiento predictivo con análisis de datos
- Integración con sistemas de notificaciones
- Análisis de tendencias y reportes

Día 10: Proyecto Final

- Diseño de un sistema de monitoreo completo
- Configuración del PLC y la comunicación en Python
- Implementación de una interfaz de usuario
- Pruebas y validación del sistema
- Documentación y mejores prácticas



Capítulo 1: Introducción a los Autómatas Siemens y Python

1.1 ¿Qué es un autómata programable (PLC)?

Un **autómata programable (PLC)** es un dispositivo electrónico diseñado para controlar procesos industriales de manera automática. Su función principal es recibir señales de entrada (sensores, botones, interruptores), procesarlas mediante un programa lógico y activar salidas (motores, válvulas, luces) para ejecutar una tarea específica.

Los PLCs se utilizan en fábricas, sistemas de control de tráfico, plantas de energía, entre otros. Son robustos y confiables, diseñados para operar en entornos industriales con condiciones adversas.

1.2 Tipos de PLC Siemens

Siemens es una de las marcas líderes en la industria de automatización y ofrece varias familias de PLCs:

- **S7-200:** Pequeño, económico, adecuado para aplicaciones simples.
- **S7-300:** Modular, usado en automatización de nivel medio.
- **S7-400:** Alto rendimiento, ideal para procesos complejos.
- **S7-1200:** Compacto y moderno, con comunicación integrada.
- **S7-1500:** Más potente y rápido, con herramientas avanzadas de diagnóstico.

Cada uno de estos PLCs tiene diferentes capacidades en cuanto a memoria, procesamiento y conectividad.

Aquí tienes una descripción más detallada y técnica de los diferentes **tipos de PLC Siemens**, incluyendo sus características técnicas, precios aproximados, lenguajes de programación y métodos de comunicación.



Tipos de PLC Siemens

Siemens es líder en la industria de la automatización y ofrece distintas gamas de **Controladores Lógicos Programables (PLC)** para cubrir necesidades desde pequeños procesos hasta la automatización industrial de alto nivel.

Clasificación de los PLC Siemens

Modelo	Tamaño	Modularidad	Aplicaciones	Nº de entradas/salidas (básico)	Velocidad de procesamiento	Precio Aproximado (Euros)
S7-200	Compacto	No	Pequeñas máquinas y sistemas básicos	6 a 24 entradas/salidas digitales	0.8 – 1.2 ms/ciclo	200 - 500
S7-300	Modular	Sí	Producción y manufactura	8 a 1024 (depende de módulos)	0.5 – 0.6 ms/ciclo	500 - 3000
S7-400	Modular	Sí	Procesos industriales complejos	16 a 4096 (expansible)	0.1 – 0.3 ms/ciclo	3000 - 10000
S7-1200	Compacto	Sí	Automatización moderna, IoT	6 a 64 E/S digitales y analógicas	0.05 – 0.1 ms/ciclo	250 - 1200
S7-1500	Modular	Sí	Industria 4.0, fábricas inteligentes	8 a 4096 (depende del CPU)	0.01 – 0.05 ms/ciclo	1000 - 15000

⚠ **Nota:** Los precios varían según el modelo, la configuración y la región.



Descripción Detallada de Cada PLC Siemens

◆ S7-200 (Compacto y económico)

- **Uso principal:** Aplicaciones simples de automatización en maquinaria pequeña.
- **Comunicación:** Soporta **PPI (Point to Point Interface)**, **Modbus RTU**, **PROFIBUS (con módulos adicionales)**.
- **Lenguajes de programación:** Ladder (LAD), Lista de instrucciones (IL).
- **Software:** **STEP 7 Micro/WIN**.
- **Expansibilidad:** Baja, pero permite módulos de comunicación y E/S adicionales.
- **Alternativa:** Fue reemplazado por el **LOGO! 8** o **S7-1200**.

◆ S7-300 (Modular y versátil)

- **Uso principal:** Automatización en **líneas de producción**, estaciones de ensamblaje y control de máquinas industriales.
- **Comunicación:** Soporta **MPI**, **PROFIBUS DP**, **PROFINET**, **Modbus TCP/IP**.
- **Lenguajes de programación:** Ladder (LAD), Lista de instrucciones (IL), FBD (Diagramas de Bloques de Función), SCL (Structured Control Language).
- **Software:** **STEP 7 en SIMATIC Manager o TIA Portal**.
- **Expansibilidad:** Alta, permite módulos de E/S digitales, analógicas, comunicación y seguridad.
- **Alternativa:** Se está reemplazando por **S7-1500**.

◆ S7-400 (Alto rendimiento para automatización avanzada)

- **Uso principal:** Control de **procesos industriales grandes**, como **planta de producción automotriz, química y sistemas SCADA**.
- **Comunicación:** Soporta **PROFIBUS DP**, **PROFINET**, **Ethernet Industrial**, **Modbus TCP**.
- **Lenguajes de programación:** LAD, IL, FBD, SCL, Grafcet (S7 Graph).
- **Software:** **STEP 7 en SIMATIC Manager**.
- **Expansibilidad:** Máxima, hasta **4096 E/S** con módulos de CPU redundantes.
- **Alternativa:** Se está reemplazando por **S7-1500** y **PCS7**.



◆ S7-1200 (Compacto y moderno con IoT)

- **Uso principal:** Automatización **pequeña y mediana**, control de **energía, HVAC, bombas y sistemas de IoT**.
- **Comunicación:** Incluye **PROFINET**, pero soporta **Modbus TCP/IP, MQTT, OPC UA** con módulos adicionales.
- **Lenguajes de programación:** LAD, FBD, SCL.
- **Software:** **TIA Portal** (Totally Integrated Automation).
- **Expansibilidad:** Alta, módulos de comunicación y entradas/salidas disponibles.
- **Ventajas:** Compatible con **Industry 4.0**, conexión a la nube y HMI.

◆ S7-1500 (El más potente y avanzado)

- **Uso principal:** Automatización **avanzada**, fábricas inteligentes, control de robots y sistemas de alta velocidad.
- **Comunicación:** Incluye **PROFINET, PROFIBUS DP, OPC UA, MQTT y Ethernet Industrial**.
- **Lenguajes de programación:** LAD, FBD, SCL, STL (Structured Text), GRAFCET.
- **Software:** **TIA Portal con capacidades avanzadas de simulación y diagnóstico**.
- **Expansibilidad:** Máxima, con módulos para redundancia y control de seguridad.
- **Ventajas:** **Mayor velocidad de procesamiento, interfaz web integrada, conectividad IoT y ciberseguridad avanzada**.

Lenguajes de Programación en PLC Siemens

Los PLC Siemens pueden programarse en varios lenguajes según la norma **IEC 61131-3**:

Lenguaje	Descripción	Uso en Siemens
Ladder (LAD)	Diagrama de contactos, basado en relés	S7-200, S7-300, S7-400, S7-1200, S7-1500
FBD (Function Block Diagram)	Uso de bloques de función	S7-300, S7-400, S7-1200, S7-1500
SCL (Structured Control Language)	Similar a Pascal, útil para cálculos complejos	S7-300, S7-400, S7-1200, S7-1500
STL (Statement List)	Programación en bajo nivel, similar a ensamblador	S7-300, S7-400, S7-1500
Grafcet (S7 Graph)	Programación por pasos y transiciones	S7-400, S7-1500



Software de Programación Siemens

Software	Compatible con	Características
STEP 7 Micro/WIN	S7-200	Básico, solo Ladder (LAD)
SIMATIC Manager (STEP 7 v5.x)	S7-300, S7-400	Software clásico para entornos industriales
TIA Portal (STEP 7 Professional)	S7-1200, S7-1500	Plataforma moderna con simulación y diagnóstico avanzado

Los PLC Siemens se adaptan a distintas necesidades industriales, desde pequeñas automatizaciones hasta fábricas inteligentes. La elección dependerá de la cantidad de entradas/salidas, velocidad de procesamiento y capacidades de comunicación. Para nuevos proyectos, se recomienda **S7-1200 para automatización compacta** y **S7-1500 para sistemas avanzados**.

1.3 Beneficios de Usar Python para Comunicación con PLCs

Python se ha convertido en una herramienta poderosa para interactuar con PLCs gracias a su facilidad de uso y la existencia de librerías como `snap7`. Algunas ventajas de usar Python incluyen:

- **Flexibilidad:** Permite integrar datos del PLC con bases de datos, visualización en dashboards y aplicaciones web.
- **Automatización:** Facilita el desarrollo de scripts para supervisión y control remoto.
- **Interoperabilidad:** Compatible con otros sistemas, como IoT, bases de datos y servicios en la nube.
- **Comunidad activa:** Existen numerosos recursos y documentación sobre su uso en la industria.

1.4 Instalación del Entorno de Desarrollo

Antes de comenzar a programar, necesitamos instalar Python y algunas librerías necesarias:

1.4.1 Instalación de Python

Si aún no tienes Python instalado, descárgalo desde la página oficial:

<https://www.python.org/downloads/>

Recomiendo la versión **3.8 o superior**.

Después de instalar, verifica la versión en la terminal o consola con:

```
python --version
```



1.4.2 Instalación de snap7

snap7 es una librería que permite la comunicación entre Python y PLCs Siemens. Para instalarla, ejecuta:

```
pip install python-snap7
```

También necesitarás descargar la biblioteca binaria de **Snap7** desde:

<http://snap7.sourceforge.net/>

Coloca los archivos en la misma carpeta del proyecto.

1.4.3 Instalación de otras librerías útiles

Para trabajar con bases de datos, gráficos y otras funcionalidades, puedes instalar:

```
pip install pandas matplotlib flask mysql-connector-python
```

1.5 Primeros Pasos con snap7

Vamos a probar si la instalación fue correcta creando un pequeño script para verificar la conexión con un PLC.

1. Importamos la librería:

```
import snap7
```

2. Creamos un cliente para conectarnos al PLC:

```
plc = snap7.client.Client()
```

3. Intentamos conectarnos a un PLC en la IP 192.168.1.100 con el rack y slot típicos de un S7-1200 (0 y 1):

```
plc.connect('192.168.1.100', 0, 1)
```

4. Verificamos el estado de la conexión:

```
if plc.get_connected():  
    print("Conexión exitosa con el PLC Siemens.")  
else:  
    print("Error en la conexión.")
```



5. Cerramos la conexión al terminar:

```
plc.disconnect()
```

Guarda este script como `test_plc.py` y ejecútalo con:

```
python test_plc.py
```

Si la conexión es exitosa, significa que tu entorno está listo para programar con PLCs Siemens.

Conclusión

En este primer capítulo, aprendimos qué es un PLC, los diferentes modelos de Siemens, y cómo Python nos ayuda a interactuar con ellos. También instalamos y probamos `snap7` para conectarnos a un PLC.

En el próximo capítulo, exploraremos cómo **leer y escribir datos en un PLC Siemens usando Python**.



Capítulo 2: Comunicación con un PLC Siemens usando Python

En este capítulo aprenderemos a establecer comunicación con un PLC Siemens utilizando Python y la librería `snap7`. Exploraremos cómo leer y escribir datos en el PLC, entenderemos el direccionamiento de memoria y realizaremos pruebas básicas.

2.1 ¿Qué es el protocolo S7?

El protocolo **S7** es un protocolo de comunicación desarrollado por Siemens para la transferencia de datos entre PLCs y otros dispositivos. Permite la lectura y escritura de datos en diferentes áreas de memoria del PLC, como:

- **Entradas (I)** → Sensores, pulsadores.
- **Salidas (Q)** → Actuadores, motores.
- **Marcas (M)** → Variables temporales dentro del PLC.
- **Bloques de datos (DB)** → Almacenan información personalizada.

Para interactuar con estas áreas, usaremos `snap7`, que actúa como un cliente S7.

2.2 Conectando Python con un PLC Siemens usando `snap7`

Antes de leer o escribir datos, necesitamos establecer una conexión con el PLC.

2.2.1 Creación de un cliente `snap7`

Vamos a crear un script en Python para conectarnos a un PLC Siemens.

Código: Conexión al PLC

```
import snap7

# Crear el cliente para comunicarse con el PLC
plc = snap7.client.Client()

# Conectar con el PLC (IP, rack, slot)
plc.connect('192.168.1.100', 0, 1)

# Verificar si la conexión fue exitosa
if plc.get_connected():
    print("Conexión establecida con el PLC Siemens.")
```



```
else:
    print("Error al conectar con el PLC.")

# Cerrar la conexión al finalizar
plc.disconnect()
```

Explicación del código:

1. Importamos `snap7` y creamos un cliente.
2. Usamos `connect(IP, rack, slot)`, donde:
 - o **IP** → Dirección del PLC en la red.
 - o **rack y slot** → Dependen del hardware (para S7-1200 y S7-1500 suelen ser 0 y 1).
3. Comprobamos si la conexión fue exitosa.
4. Desconectamos el cliente al terminar.

Guarda este código como `conexion_plc.py` y ejecútalo con:

```
python conexion_plc.py
```

Si el PLC está configurado correctamente, verás:

```
Conexión establecida con el PLC Siemens.
```

De lo contrario, revisa la IP y la configuración de comunicación en **TIA Portal**.

2.3 Lectura de Datos desde el PLC

Ahora aprenderemos a leer valores de **entradas, salidas y bloques de datos**.

2.3.1 Lectura de Entradas Digitales

Las **entradas digitales (I)** almacenan valores **0 o 1** (apagado/encendido). Podemos leerlas con:

```
import snap7

def leer_entrada(plc, byte, bit):
    # Leer el estado de la entrada
    area = snap7.types.Areas.PE # PE = Process Inputs (entradas)
    data = plc.read_area(area, 0, byte, 1) # Leer 1 byte
    valor = (data[0] >> bit) & 1 # Extraer el bit deseado
    return valor

# Conectar al PLC
plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)
```



```
# Leer la entrada I0.2
estado = leer_entrada(plc, 0, 2)
print(f"Estado de I0.2: {estado}")

plc.disconnect()
```

Explicación:

- `snap7.types.Areas.PE` indica que queremos leer entradas (I).
- `read_area(area, DB, byte, size)` obtiene datos del PLC.
- Usamos operaciones de bits para extraer el valor de una entrada específica.

2.3.2 Lectura de una Salida Digital

Las **salidas digitales (Q)** se manejan igual que las entradas. Para leer la salida **Q0.1**:

```
def leer_salida(plc, byte, bit):
    area = snap7.types.Areas.PA # PA = Process Outputs (salidas)
    data = plc.read_area(area, 0, byte, 1)
    return (data[0] >> bit) & 1

# Leer Q0.1
estado = leer_salida(plc, 0, 1)
print(f"Estado de Q0.1: {estado}")
```

2.3.3 Lectura de un Bloque de Datos (DB)

Los **bloques de datos (DB)** almacenan información personalizada, como valores de temperatura o contadores.

Ejemplo: **Leer un número entero (INT) desde DB1, byte 4:**

```
def leer_DB_int(plc, db_number, start_byte):
    data = plc.read_area(snap7.types.Areas.DB, db_number, start_byte, 2)
    valor = int.from_bytes(data, byteorder='big', signed=True) # Convertir a
    INT
    return valor

# Leer valor de DB1, byte 4
valor = leer_DB_int(plc, 1, 4)
print(f"Valor leído del DB1: {valor}")
```



2.4 Escritura de Datos en el PLC

Ahora aprenderemos a escribir valores en el PLC.

2.4.1 Escritura en una Salida Digital (Q)

Para activar la salida **Q0.1**, usamos `write_area()`:

```
def escribir_salida(plc, byte, bit, estado):
    area = snap7.types.Areas.PA # Salidas
    data = plc.read_area(area, 0, byte, 1)
    if estado:
        data = bytearray([data[0] | (1 << bit)]) # Encender bit
    else:
        data = bytearray([data[0] & ~(1 << bit)]) # Apagar bit
    plc.write_area(area, 0, byte, data)

# Activar Q0.1
escribir_salida(plc, 0, 1, 1)
print("Q0.1 activada")
```

2.4.2 Escritura en un Bloque de Datos (DB)

Para escribir un número **20** en DB1, byte 4:

```
def escribir_DB_int(plc, db_number, start_byte, valor):
    data = valor.to_bytes(2, byteorder='big', signed=True)
    plc.write_area(snap7.types.Areas.DB, db_number, start_byte, data)

# Escribir 20 en DB1, byte 4
escribir_DB_int(plc, 1, 4, 20)
print("Valor 20 escrito en DB1")
```

2.5 Manejo de Errores en la Comunicación

Cuando trabajamos con PLCs, pueden ocurrir errores de conexión. Para evitar que el programa se detenga, usamos excepciones:

```
try:
    plc.connect('192.168.1.100', 0, 1)
    print("Conexión exitosa")
except snap7.exceptions.Snap7Exception as e:
    print(f"Error de conexión: {e}")
finally:
    plc.disconnect()
```



2.6 Conclusión

En este capítulo aprendimos a: ✓ Conectarnos al PLC.

✓ Leer y escribir entradas, salidas y bloques de datos.

✓ Manejar errores de comunicación.

En el **próximo capítulo**, profundizaremos en la manipulación avanzada de datos dentro del PLC usando Python.



Capítulo 3: Lectura y Escritura de Variables en el PLC

En este capítulo profundizaremos en el manejo de datos en los PLC Siemens. Veremos cómo leer y escribir en diferentes áreas de memoria, entenderemos el direccionamiento en detalle y aprenderemos a manejar datos más complejos, como números enteros, flotantes y cadenas de texto.

3.1 Direccionamiento en la Memoria del PLC

Para comunicarnos con un PLC Siemens, es fundamental conocer cómo se organizan los datos en su memoria. Los principales tipos de memoria en los PLC Siemens son:

- **Entradas Digitales (I)** → Sensores, pulsadores (0 o 1).
- **Salidas Digitales (Q)** → Motores, relés (0 o 1).
- **Memoria de Marcas (M)** → Variables internas del PLC.
- **Bloques de Datos (DB)** → Almacenan valores más complejos (INT, REAL, STRING).
- **Contadores y Temporizadores (C y T)** → Para operaciones de control.

Cada tipo de memoria tiene una dirección específica. Por ejemplo:

- **I0.1** → Entrada digital en el byte 0, bit 1.
- **Q1.3** → Salida digital en el byte 1, bit 3.
- **M10.5** → Marca en el byte 10, bit 5.
- **DB1.DB4** → Valor de 4 bytes en el DB1, byte 4.

3.2 Leyendo y Escribiendo Variables Digitales

Las variables digitales almacenan valores **0** o **1**. Para interactuar con ellas, usamos `read_area()` y `write_area()`.

3.2.1 Leer una Entrada Digital

Vamos a leer el estado de una entrada digital **I0.2**.

```
import snap7

def leer_entrada(plc, byte, bit):
    area = snap7.types.Areas.PE # PE = Process Inputs
    data = plc.read_area(area, 0, byte, 1) # Leer 1 byte
```



```
        return (data[0] >> bit) & 1 # Extraer el bit

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

estado = leer_entrada(plc, 0, 2)
print(f"Estado de I0.2: {estado}")

plc.disconnect()
```

Explicación:

- `read_area(snap7.types.Areas.PE, 0, byte, 1)` obtiene el byte de entrada.
- `(data[0] >> bit) & 1` extrae el bit específico.

3.2.2 Escribir en una Salida Digital

Para activar **Q0.1**, usamos `write_area()`.

```
def escribir_salida(plc, byte, bit, estado):
    area = snap7.types.Areas.PA # PA = Process Outputs
    data = plc.read_area(area, 0, byte, 1)
    if estado:
        data = bytearray([data[0] | (1 << bit)]) # Encender bit
    else:
        data = bytearray([data[0] & ~(1 << bit)]) # Apagar bit
    plc.write_area(area, 0, byte, data)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

escribir_salida(plc, 0, 1, 1) # Encender Q0.1
print("Q0.1 activada")

plc.disconnect()
```

3.3 Leyendo y Escribiendo Variables Analógicas

Las variables analógicas pueden almacenar números enteros o flotantes.

3.3.1 Leer un Número Entero (INT) desde un DB

```
def leer_DB_int(plc, db_number, start_byte):
    data = plc.read_area(snap7.types.Areas.DB, db_number, start_byte, 2)
    return int.from_bytes(data, byteorder='big', signed=True)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)
```



```
valor = leer_DB_int(plc, 1, 4) # Leer un INT en DB1, byte 4
print(f"Valor leído: {valor}")

plc.disconnect()
```

Explicación:

- Se leen **2 bytes** desde DB1, a partir del byte 4.
- `int.from_bytes(data, byteorder='big', signed=True)` convierte los bytes a entero.

3.3.2 Escribir un Número Entero (INT) en un DB

```
def escribir_DB_int(plc, db_number, start_byte, valor):
    data = valor.to_bytes(2, byteorder='big', signed=True)
    plc.write_area(snap7.types.Areas.DB, db_number, start_byte, data)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

escribir_DB_int(plc, 1, 4, 100) # Escribir 100 en DB1, byte 4
print("Valor escrito en DB1")

plc.disconnect()
```

3.3.3 Leer un Número Flotante (REAL) desde un DB

Los valores REAL ocupan **4 bytes** en el PLC.

```
import struct

def leer_DB_real(plc, db_number, start_byte):
    data = plc.read_area(snap7.types.Areas.DB, db_number, start_byte, 4)
    return struct.unpack('>f', data)[0] # Convertir bytes a float

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

valor = leer_DB_real(plc, 1, 8) # Leer un REAL en DB1, byte 8
print(f"Valor REAL leído: {valor}")

plc.disconnect()
```



3.3.4 Escribir un Número Flotante (REAL) en un DB

```
def escribir_DB_real(plc, db_number, start_byte, valor):
    data = struct.pack('>f', valor)
    plc.write_area(snap7.types.Areas.DB, db_number, start_byte, data)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

escribir_DB_real(plc, 1, 8, 12.34) # Escribir 12.34 en DB1, byte 8
print("Valor REAL escrito en DB1")

plc.disconnect()
```

3.4 Leyendo y Escribiendo Cadenas de Texto

Los PLC Siemens permiten almacenar **cadenas de texto (STRING)** en bloques de datos.

3.4.1 Leer una Cadena de Texto desde un DB

```
def leer_DB_string(plc, db_number, start_byte, length):
    data = plc.read_area(snap7.types.Areas.DB, db_number, start_byte, length)
    return data[2:].decode('utf-8').strip('\x00')

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

texto = leer_DB_string(plc, 1, 12, 20) # Leer STRING en DB1, byte 12
print(f"Texto leído: {texto}")

plc.disconnect()
```

3.4.2 Escribir una Cadena de Texto en un DB

```
def escribir_DB_string(plc, db_number, start_byte, texto, max_length):
    data = bytearray(max_length + 2) # Reservar espacio
    data[0] = max_length # Longitud máxima
    data[1] = len(texto) # Longitud actual
    data[2:2+len(texto)] = texto.encode('utf-8') # Insertar texto
    plc.write_area(snap7.types.Areas.DB, db_number, start_byte, data)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

escribir_DB_string(plc, 1, 12, "Hola PLC", 20)
print("Texto escrito en DB1")

plc.disconnect()
```



3.5 Conclusión

En este capítulo aprendimos a: ✓ Leer y escribir variables digitales (0 y 1).

✓ Leer y escribir enteros, flotantes y cadenas en bloques de datos.

✓ Manejar correctamente la memoria del PLC en Python.

En el **próximo capítulo**, exploraremos cómo monitorear y controlar el PLC en **tiempo real**.



Capítulo 4: Manejo Avanzado de Datos en Python y PLC

En este capítulo aprenderemos técnicas avanzadas para manejar datos en un PLC Siemens usando Python. Nos enfocaremos en la conversión de datos, estructuras de almacenamiento y comunicación en tiempo real.

4.1 Tipos de Datos en PLC Siemens y su Conversión en Python

Los PLCs Siemens manejan varios tipos de datos que deben convertirse correctamente para su uso en Python:

Tipo en PLC	Bytes	Tipo en Python
BOOL	1 bit	bool
BYTE	1 byte	int
INT	2 bytes	int
DINT	4 bytes	int
REAL	4 bytes	float
STRING[n]	n+2 bytes	str

Python maneja estos datos usando `struct` y `int.from_bytes()`. Veamos ejemplos.

4.2 Conversión de Datos entre Python y PLC

Cuando leemos o escribimos datos en el PLC, es esencial realizar conversiones de datos.

4.2.1 Convertir Bytes a Enteros

```
def bytes_a_int(data):  
    return int.from_bytes(data, byteorder='big', signed=True)  
  
# Ejemplo  
dato_plc = b'\x00\x64' # 100 en binario (INT en el PLC)  
valor = bytes_a_int(dato_plc)  
print(f"Valor convertido: {valor}")
```



4.2.2 Convertir Enteros a Bytes

```
def int_a_bytes(valor):  
    return valor.to_bytes(2, byteorder='big', signed=True)  
  
# Ejemplo  
dato_bytes = int_a_bytes(100)  
print(f"Bytes convertidos: {dato_bytes}")
```

4.2.3 Convertir Bytes a Flotante (REAL)

```
import struct  
  
def bytes_a_real(data):  
    return struct.unpack('>f', data)[0]  
  
# Ejemplo  
dato_plc = b'\x41\x48\x00\x00' # 12.5 en IEEE 754  
valor = bytes_a_real(dato_plc)  
print(f"Valor REAL convertido: {valor}")
```

4.2.4 Convertir Flotante a Bytes

```
def real_a_bytes(valor):  
    return struct.pack('>f', valor)  
  
# Ejemplo  
dato_bytes = real_a_bytes(12.5)  
print(f"Bytes convertidos: {dato_bytes}")
```

4.3 Uso de Estructuras y Listas en Python para Datos del PLC

Al leer múltiples valores del PLC, podemos almacenarlos en listas y diccionarios para facilitar su manejo.

4.3.1 Leer Múltiples Datos y Guardarlos en una Lista

```
plc = snap7.client.Client()  
plc.connect('192.168.1.100', 0, 1)  
  
datos = []  
for i in range(0, 10, 2): # Leer 5 valores INT desde DB1  
    valor = leer_DB_int(plc, 1, i)  
    datos.append(valor)  
  
print("Valores leídos:", datos)  
plc.disconnect()
```



4.3.2 Guardar Datos en un Diccionario para un Reporte

```
reporte = {
    "Temperatura": leer_DB_real(plc, 1, 0),
    "Presión": leer_DB_real(plc, 1, 4),
    "Velocidad": leer_DB_real(plc, 1, 8),
}

print("Reporte de sensores:", reporte)
```

4.4 Comunicación en Tiempo Real con el PLC

Para monitorear datos en tiempo real, podemos usar un **bucle infinito** con `time.sleep()`.

4.4.1 Monitoreo de Variables en Tiempo Real

```
import time

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

try:
    while True:
        temp = leer_DB_real(plc, 1, 0)
        presion = leer_DB_real(plc, 1, 4)
        print(f"Temperatura: {temp}°C | Presión: {presion} Bar")
        time.sleep(1) # Leer cada 1 segundo
except KeyboardInterrupt:
    print("Monitoreo detenido.")

plc.disconnect()
```

4.5 Depuración de Errores en la Comunicación

Es importante capturar errores para evitar bloqueos en el sistema.

```
try:
    plc.connect('192.168.1.100', 0, 1)
    temp = leer_DB_real(plc, 1, 0)
    print(f"Temperatura: {temp}°C")
except snap7.exceptions.Snap7Exception as e:
    print(f"Error en la comunicación: {e}")
finally:
    plc.disconnect()
```



4.6 Conclusión

En este capítulo aprendimos a: ✓ Convertir datos entre Python y PLC.

✓ Usar estructuras como listas y diccionarios para organizar datos.

✓ Monitorear variables en tiempo real.

✓ Manejar errores en la comunicación.

En el **próximo capítulo**, implementaremos **un sistema de monitoreo y control en tiempo real con Python**.



Capítulo 5: Monitoreo y Control en Tiempo Real

En este capítulo, desarrollaremos un sistema de monitoreo en tiempo real para leer y controlar variables del PLC Siemens con Python. Aprenderemos a visualizar datos en la consola y en una interfaz gráfica simple.

5.1 Implementación de un Sistema de Monitoreo en Python

Para monitorear valores en tiempo real, usaremos un bucle que lea los datos del PLC y los muestre continuamente.

5.1.1 Código para Monitorear Sensores en la Consola

```
import snap7
import time

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

try:
    while True:
        temp = leer_DB_real(plc, 1, 0)
        presion = leer_DB_real(plc, 1, 4)
        velocidad = leer_DB_real(plc, 1, 8)
        print(f"Temperatura: {temp:.2f}°C | Presión: {presion:.2f} Bar |
Velocidad: {velocidad:.2f} m/s")
        time.sleep(1)
except KeyboardInterrupt:
    print("Monitoreo detenido.")
finally:
    plc.disconnect()
```

Explicación:

- Se conecta al PLC y lee datos en un bucle.
- `time.sleep(1)` evita sobrecargar la CPU.
- Captura `KeyboardInterrupt` para detener el programa con `Ctrl+C`.



5.2 Control de Salidas en Tiempo Real

Podemos activar o desactivar salidas del PLC en función de las lecturas de sensores.

5.2.1 Activar una Salida Basada en una Condición

```
def controlar_salida(plc):
    temp = leer_DB_real(plc, 1, 0)
    if temp > 50.0: # Si la temperatura supera 50°C, activar Q0.1
        escribir_salida(plc, 0, 1, 1)
        print("¡Alerta! Temperatura alta. Activando ventilador.")
    else:
        escribir_salida(plc, 0, 1, 0)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

try:
    while True:
        controlar_salida(plc)
        time.sleep(2)
except KeyboardInterrupt:
    print("Control detenido.")
finally:
    plc.disconnect()
```

5.3 Registro de Eventos en un Log

Registrar eventos es útil para analizar datos históricos.

5.3.1 Guardar Lecturas en un Archivo

```
def registrar_evento(mensaje):
    with open("registro_plc.txt", "a") as log:
        log.write(f"{time.strftime('%Y-%m-%d %H:%M:%S')} - {mensaje}\n")

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

try:
    while True:
        temp = leer_DB_real(plc, 1, 0)
        presion = leer_DB_real(plc, 1, 4)
        registrar_evento(f"Temperatura: {temp:.2f}°C, Presión: {presion:.2f} Bar")
        time.sleep(5)
except KeyboardInterrupt:
    print("Registro detenido.")
finally:
    plc.disconnect()
```



Explicación:

- `time.strftime('%Y-%m-%d %H:%M:%S')` genera una marca de tiempo.
- Los datos se almacenan en `registro_plc.txt`.

5.4 Creación de una Interfaz Gráfica con Tkinter

Para visualizar los datos de forma más atractiva, crearemos una interfaz con **Tkinter**.

5.4.1 Código para una Interfaz Simple

```
import tkinter as tk
from tkinter import Label
import threading

def actualizar_datos():
    temp = leer_DB_real(plc, 1, 0)
    presion = leer_DB_real(plc, 1, 4)
    lbl_temp.config(text=f"Temperatura: {temp:.2f}°C")
    lbl_presion.config(text=f"Presión: {presion:.2f} Bar")
    root.after(1000, actualizar_datos) # Actualiza cada segundo

# Configuración de la interfaz
root = tk.Tk()
root.title("Monitor PLC Siemens")

lbl_temp = Label(root, text="Temperatura: --°C", font=("Arial", 14))
lbl_temp.pack()
lbl_presion = Label(root, text="Presión: -- Bar", font=("Arial", 14))
lbl_presion.pack()

# Conectar al PLC
plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

# Iniciar la actualización de datos
actualizar_datos()

# Iniciar la interfaz
root.mainloop()

# Desconectar al cerrar
plc.disconnect()
```

Explicación:

- Usa **Tkinter** para crear una ventana con etiquetas de datos.
- `actualizar_datos()` actualiza los valores cada segundo.
- Se ejecuta en un hilo separado para no bloquear la interfaz.



5.5 Implementación de Botones de Control

Podemos agregar botones para activar o desactivar salidas del PLC.

5.5.1 Código con Botones para Controlar Salidas

```
def activar_salida():
    escribir_salida(plc, 0, 1, 1)
    lbl_estado.config(text="Salida Q0.1: ACTIVADA")

def desactivar_salida():
    escribir_salida(plc, 0, 1, 0)
    lbl_estado.config(text="Salida Q0.1: DESACTIVADA")

# Configuración de la interfaz
root = tk.Tk()
root.title("Control PLC Siemens")

lbl_temp = Label(root, text="Temperatura: --°C", font=("Arial", 14))
lbl_temp.pack()
lbl_presion = Label(root, text="Presión: -- Bar", font=("Arial", 14))
lbl_presion.pack()

btn_on = tk.Button(root, text="Activar Salida", command=activar_salida)
btn_on.pack()
btn_off = tk.Button(root, text="Desactivar Salida",
                    command=desactivar_salida)
btn_off.pack()

lbl_estado = Label(root, text="Salida Q0.1: --", font=("Arial", 14))
lbl_estado.pack()

actualizar_datos()
root.mainloop()
```

5.6 Conclusión

En este capítulo hemos aprendido a: ✓ Monitorear valores del PLC en **tiempo real**.

✓ **Activar y desactivar** salidas según condiciones.

✓ Registrar eventos en **archivos de log**.

✓ Crear una **interfaz gráfica** con Tkinter para visualizar y controlar datos.

En el **próximo capítulo**, aprenderemos a **integrar Python con TIA Portal y PLCSIM** para simular un PLC Siemens.



Capítulo 6: Programación de PLCs con TIA Portal y Python

En este capítulo aprenderemos cómo integrar **Python con TIA Portal**, el entorno de desarrollo de Siemens, y cómo utilizar **PLCSIM** para simular un PLC sin hardware físico. Esto nos permitirá probar nuestros programas sin necesidad de un PLC real.

6.1 Introducción a TIA Portal

TIA Portal (Totally Integrated Automation Portal) es el software de Siemens para programar y configurar PLCs como **S7-1200** y **S7-1500**. Con él podemos: ✓ Diseñar programas en **Ladder (LAD)**, **FBD** y **SCL**.

- ✓ Configurar dispositivos y redes industriales.
- ✓ Crear interfaces HMI.
- ✓ Simular la lógica del PLC con **PLCSIM**.

6.1.1 Descarga e Instalación de TIA Portal

Puedes obtener TIA Portal desde la página oficial de Siemens. Hay versiones de prueba disponibles.

6.2 Configuración de un Proyecto en TIA Portal

Para integrar Python con un PLC Siemens en TIA Portal, seguimos estos pasos:

1. **Abrir TIA Portal** y crear un nuevo proyecto.
2. **Agregar un PLC Siemens** (por ejemplo, S7-1200).
3. **Configurar la dirección IP** del PLC (Ejemplo: 192.168.1.100).
4. **Crear un DB (Bloque de Datos)** para almacenar variables.
5. **Configurar PLCSIM** para probar la comunicación sin hardware.



6.3 Uso de PLCSIM para Simulación

PLCSIM permite probar programas de PLC sin un equipo físico.

6.3.1 Pasos para Usar PLCSIM

1. **Abrir PLCSIM** y cargar el programa desde TIA Portal.
2. **Configurar la IP virtual del PLC** (Ejemplo: 192.168.1.101).
3. **Poner el PLC en modo RUN** para ejecutar la simulación.
4. **Comprobar la conexión desde Python** usando `snap7`.

```
import snap7

plc = snap7.client.Client()
plc.connect('192.168.1.101', 0, 1) # IP del PLC en PLCSIM

if plc.get_connected():
    print("Conectado a PLCSIM correctamente.")
else:
    print("No se pudo conectar.")

plc.disconnect()
```

6.4 Lectura y Escritura en el PLC Simulado

Podemos leer y escribir valores en la simulación como si fuera un PLC real.

6.4.1 Leer un Valor del DB en PLCSIM

```
def leer_DB_real(plc, db_number, start_byte):
    import struct
    data = plc.read_area(snap7.types.Areas.DB, db_number, start_byte, 4)
    return struct.unpack('>f', data)[0]

plc = snap7.client.Client()
plc.connect('192.168.1.101', 0, 1)

valor = leer_DB_real(plc, 1, 0) # Leer REAL en DB1
print(f"Valor leído: {valor}")

plc.disconnect()
```



6.4.2 Escribir un Valor en un DB Simulado

```
def escribir_DB_real(plc, db_number, start_byte, valor):
    import struct
    data = struct.pack('>f', valor)
    plc.write_area(snap7.types.Areas.DB, db_number, start_byte, data)

plc = snap7.client.Client()
plc.connect('192.168.1.101', 0, 1)

escribir_DB_real(plc, 1, 0, 25.5) # Escribir 25.5 en DB1
print("Valor escrito en la simulación.")

plc.disconnect()
```

6.5 Pruebas de Integración con Python

Ahora podemos simular sensores y actuar sobre variables.

6.5.1 Simular un Sensor de Temperatura en PLCSIM

1. **Abrir el DB en TIA Portal.**
2. **Modificar manualmente el valor** para simular un sensor.
3. **Leer el valor desde Python** con el código anterior.

6.5.2 Controlar una Salida Simulada

```
def controlar_salida_simulada(plc):
    temp = leer_DB_real(plc, 1, 0)
    if temp > 30.0:
        escribir_salida(plc, 0, 1, 1) # Activar Q0.1 en PLCSIM
        print("Salida activada en simulación.")
    else:
        escribir_salida(plc, 0, 1, 0)

plc = snap7.client.Client()
plc.connect('192.168.1.101', 0, 1)

try:
    while True:
        controlar_salida_simulada(plc)
        time.sleep(2)
except KeyboardInterrupt:
    print("Simulación detenida.")

plc.disconnect()
```



6.6 Conclusión

- ✓ Aprendimos a integrar **Python con TIA Portal y PLCSIM**.
- ✓ Simulamos la comunicación con un PLC sin hardware físico.
- ✓ Leímos y escribimos datos en el **DB** del PLC virtual.
- ✓ Probamos la **automatización de salidas** en un entorno simulado.

En el **próximo capítulo**, aprenderemos **cómo conectar Python con múltiples PLCs y redes industriales**.



Capítulo 7: Conectividad Avanzada y Redes Industriales

En este capítulo, aprenderemos a **comunicar Python con múltiples PLCs Siemens** y a integrar **protocolos industriales** como **MQTT** para conectar con la nube. También veremos cómo almacenar datos en una base de datos.

7.1 Comunicación con Múltiples PLCs desde Python

En entornos industriales, a menudo hay más de un PLC en la red. Python puede gestionar múltiples conexiones simultáneamente.

7.1.1 Conectar a Varios PLCs

```
import snap7

# Lista de PLCs en la red
plcs = [
    {'ip': '192.168.1.100', 'rack': 0, 'slot': 1},
    {'ip': '192.168.1.101', 'rack': 0, 'slot': 1},
]

clientes = []

# Conectar a cada PLC
for plc in plcs:
    cliente = snap7.client.Client()
    cliente.connect(plc['ip'], plc['rack'], plc['slot'])
    clientes.append(cliente)
    print(f"Conectado a {plc['ip']}")

# Leer datos de cada PLC
for i, cliente in enumerate(clientes):
    temp = leer_DB_real(cliente, 1, 0)
    print(f"PLC {i} - Temperatura: {temp:.2f}°C")

# Desconectar
for cliente in clientes:
    cliente.disconnect()
```

Explicación:

- Crea una lista de PLCs con sus IPs.
- Se conecta a cada PLC y almacena las conexiones en una lista.
- Itera sobre cada PLC para leer y mostrar valores.



7.2 Uso de MQTT para Integración con IoT

MQTT es un protocolo ligero para enviar datos a la nube o a otros dispositivos.

7.2.1 Instalación de MQTT en Python

```
pip install paho-mqtt
```

7.2.2 Publicar Datos del PLC en un Servidor MQTT

```
import paho.mqtt.client as mqtt

broker = "test.mosquitto.org"
cliente_mqtt = mqtt.Client("PLC_Siemens")

cliente_mqtt.connect(broker)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

while True:
    temp = leer_DB_real(plc, 1, 0)
    cliente_mqtt.publish("plc/temperatura", temp)
    print(f"Publicado: {temp}°C")
    time.sleep(5)
```

Explicación:

- Se conecta a **Mosquitto**, un servidor MQTT gratuito.
- **Lee la temperatura del PLC** y la envía cada 5 segundos.
- Puede visualizarse con un cliente MQTT como **MQTT Explorer**.

7.3 Conectar Python con una Base de Datos

Podemos almacenar datos del PLC en **MySQL** o **SQLite** para análisis.

7.3.1 Instalación de MySQL Connector

```
pip install mysql-connector-python
```



7.3.2 Guardar Datos del PLC en MySQL

```
import mysql.connector

conexion = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="plc_data"
)

cursor = conexion.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS registros (id INT AUTO_INCREMENT
PRIMARY KEY, temperatura FLOAT, tiempo TIMESTAMP DEFAULT CURRENT_TIMESTAMP)")

def guardar_en_db(temp):
    cursor.execute("INSERT INTO registros (temperatura) VALUES (%s)",
    (temp,))
    conexion.commit()

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

while True:
    temp = leer_DB_real(plc, 1, 0)
    guardar_en_db(temp)
    print(f"Guardado en BD: {temp}°C")
    time.sleep(5)
```

Explicación:

- Crea una **tabla en MySQL** si no existe.
- Guarda la temperatura en la base de datos cada 5 segundos.

7.4 Integración con Node-RED y Dashboards

Podemos visualizar datos en **gráficos en tiempo real** con Node-RED.

7.4.1 Enviar Datos desde Python a Node-RED

1. **Abrir Node-RED** (`node-red` en la terminal).
2. **Configurar un nodo MQTT** para recibir datos en `plc/temperatura`.
3. **Conectar el nodo MQTT a un gráfico** en el dashboard.



7.4.2 Publicar Datos desde Python

```
cliente_mqtt.publish("plc/temperatura", temp)
```

Ahora, los datos del PLC se ven en **gráficos web en tiempo real**.

7.5 Seguridad en la Comunicación con PLCs

Es importante proteger el acceso a los PLCs:

- ✓ Usar VPNs para evitar accesos no autorizados.
- ✓ Restringir accesos en el firewall del PLC.
- ✓ Encriptar datos usando TLS en MQTT.
- ✓ Usar contraseñas seguras en la red del PLC.

7.6 Conclusión

- ✓ Nos conectamos a **varios PLCs simultáneamente**.
- ✓ Enviamos datos a la nube con MQTT.
- ✓ Almacenamos registros en una base de datos MySQL.
- ✓ Visualizamos datos en tiempo real con Node-RED.

En el **próximo capítulo**, aprenderemos cómo **automatizar procesos de control remoto con Python y una interfaz web**.



Capítulo 8: Automatización y Control Remoto

En este capítulo aprenderemos cómo **controlar un PLC Siemens de forma remota** utilizando Python y una aplicación web. Implementaremos un **servidor web con Flask** para recibir comandos y enviarlos al PLC, permitiendo así el monitoreo y control desde cualquier navegador.

8.1 Creación de un Servidor Web con Flask

Flask es un framework ligero para crear servidores web en Python. Lo usaremos para enviar y recibir datos del PLC.

8.1.1 Instalación de Flask

```
pip install flask
```

8.1.2 Creación de un Servidor Flask

```
from flask import Flask, jsonify
import snap7

app = Flask(__name__)

plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

@app.route('/estado', methods=['GET'])
def obtener_estado():
    temp = leer_DB_real(plc, 1, 0)
    return jsonify({"temperatura": temp})

@app.route('/activar_salida', methods=['GET'])
def activar_salida():
    escribir_salida(plc, 0, 1, 1)
    return jsonify({"mensaje": "Salida activada"})

@app.route('/desactivar_salida', methods=['GET'])
def desactivar_salida():
    escribir_salida(plc, 0, 1, 0)
    return jsonify({"mensaje": "Salida desactivada"})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```



8.2 Control del PLC desde un Navegador

Una vez ejecutado el servidor con:

```
python servidor_plc.py
```

Podemos acceder desde un navegador:

- **Obtener temperatura:** <http://localhost:5000/estado>
- **Activar salida Q0.1:** http://localhost:5000/activar_salida
- **Desactivar salida Q0.1:** http://localhost:5000/desactivar_salida

8.3 Creación de una Interfaz Web

Podemos mejorar la interacción con una página HTML.

8.3.1 Código HTML para Control Remoto

Guarda este código como `index.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Control PLC Siemens</title>
  <script>
    function obtenerEstado() {
      fetch('/estado')
        .then(response => response.json())
        .then(data => {
          document.getElementById("temperatura").innerText =
"Temperatura: " + data.temperatura + "°C";
        });
    }

    function activarSalida() {
      fetch('/activar_salida')
        .then(response => alert("Salida activada"));
    }

    function desactivarSalida() {
      fetch('/desactivar_salida')
        .then(response => alert("Salida desactivada"));
    }
  </script>
</head>
<body>
```



```
<h1>Control de PLC Siemens</h1>
<p id="temperatura">Temperatura: --°C</p>
<button onclick="obtenerEstado()">Actualizar Temperatura</button>
<button onclick="activarSalida()">Activar Salida</button>
<button onclick="desactivarSalida()">Desactivar Salida</button>
</body>
</html>
```

8.3.2 Servir la Página con Flask

Modifica el servidor Flask para incluir la página web:

```
from flask import Flask, jsonify, send_file

@app.route('/')
def index():
    return send_file('index.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

8.4 Monitoreo y Alarmas en Tiempo Real

Podemos generar alertas si la temperatura supera un umbral.

8.4.1 Notificaciones Automáticas

```
@app.route('/alerta_temperatura', methods=['GET'])
def alerta_temperatura():
    temp = leer_DB_real(plc, 1, 0)
    if temp > 50.0:
        return jsonify({"alerta": "¡Temperatura alta!", "valor": temp})
    return jsonify({"alerta": "Normal", "valor": temp})
```

Ahora, si la temperatura es superior a 50°C, se genera una alerta.

8.5 Envío de Notificaciones por Correo

Podemos enviar un correo si la temperatura es crítica.

8.5.1 Instalación de smtplib

```
pip install yagmail
```



8.5.2 Código para Enviar Correos

```
import yagmail

def enviar_alerta(temperatura):
    yag = yagmail.SMTP("tu_correo@gmail.com", "tu_contraseña")
    yag.send(
        to="destinatario@gmail.com",
        subject="Alerta de Temperatura",
        contents=f"La temperatura del PLC es de {temperatura}°C. ¡Revisar!"
    )

@app.route('/chequear_alerta', methods=['GET'])
def chequear_alerta():
    temp = leer_DB_real(plc, 1, 0)
    if temp > 50.0:
        enviar_alerta(temp)
    return jsonify({"mensaje": "Alerta enviada"})
    return jsonify({"mensaje": "Temperatura normal"})
```

8.6 Conclusión

- ✓ Creamos un servidor **Flask** para monitoreo remoto.
- ✓ Desarrollamos una **página web** para controlar el PLC.
- ✓ Agregamos **notificaciones y alertas** en caso de fallos.
- ✓ Automatizamos el **envío de correos** si hay temperaturas peligrosas.

En el **próximo capítulo**, implementaremos **optimización del código y mantenimiento predictivo** en sistemas industriales.



Capítulo 9: Optimización y Mantenimiento Predictivo

En este capítulo aprenderemos a mejorar el rendimiento de la comunicación con el PLC, detectar fallos antes de que ocurran y registrar datos para análisis futuros.

9.1 Estrategias de Optimización en la Comunicación

Cuando trabajamos con PLCs, la eficiencia es clave. Algunas técnicas para mejorar el rendimiento incluyen:

- ✓ **Reducir la frecuencia de lecturas** innecesarias.
- ✓ **Leer varios datos en una sola solicitud** en lugar de múltiples accesos.
- ✓ **Usar caché de datos** en Python para evitar accesos repetidos.
- ✓ **Implementar hilos y asincronía** para evitar bloqueos.

9.1.1 Leer Múltiples Valores en una Solicitud

En lugar de hacer varias lecturas individuales, podemos leer un bloque de memoria de una sola vez:

```
def leer_multiples_datos(plc, db_number, start_byte, size):  
    data = plc.read_area(snap7.types.Areas.DB, db_number, start_byte, size)  
    valores = struct.unpack('>f f f', data) # Leer 3 valores flotantes  
    return valores  
  
plc = snap7.client.Client()  
plc.connect('192.168.1.100', 0, 1)  
  
temp, presion, velocidad = leer_multiples_datos(plc, 1, 0, 12)  
print(f"Temperatura: {temp}°C, Presión: {presion} Bar, Velocidad: {velocidad}  
m/s")  
  
plc.disconnect()
```

Explicación:

- **Lee 12 bytes de una vez** en lugar de hacer 3 lecturas separadas.
- Se descomponen los valores con `struct.unpack()` para mayor eficiencia.



9.2 Diagnóstico de Fallos en el PLC desde Python

Podemos detectar errores de conexión y fallos en sensores.

9.2.1 Detectar Fallo en la Comunicación

```
def verificar_conexion(plc):  
    try:  
        plc.get_cpu_state()  
        return "PLC conectado"  
    except snap7.exceptions.Snap7Exception:  
        return "Error: No se puede conectar al PLC"  
  
plc = snap7.client.Client()  
plc.connect('192.168.1.100', 0, 1)  
print(verificar_conexion(plc))  
plc.disconnect()
```

9.3 Implementación de Mantenimiento Predictivo

El mantenimiento predictivo permite **anticipar fallos** en sensores y equipos a partir del análisis de datos históricos.

9.3.1 Registro de Datos en CSV

Podemos registrar datos del PLC en un archivo CSV para su análisis.

```
import csv  
import time  
  
def registrar_datos(temp, presion, velocidad):  
    with open("datos_plc.csv", "a", newline='') as archivo:  
        escritor = csv.writer(archivo)  
        escritor.writerow([time.strftime('%Y-%m-%d %H:%M:%S'), temp, presion,  
velocidad])  
  
plc = snap7.client.Client()  
plc.connect('192.168.1.100', 0, 1)  
  
while True:  
    temp, presion, velocidad = leer_multiples_datos(plc, 1, 0, 12)  
    registrar_datos(temp, presion, velocidad)  
    print(f"Datos registrados: {temp}°C, {presion} Bar, {velocidad} m/s")  
    time.sleep(5)
```

Explicación:

- Cada 5 segundos, guarda los datos en `datos_plc.csv`.
- Este archivo se puede analizar con **Excel o Python**.



9.3.2 Detectar Patrones Anómalos con Python

Podemos usar estadísticas para detectar anomalías en los datos del PLC.

```
import pandas as pd

df = pd.read_csv("datos_plc.csv", names=["Tiempo", "Temperatura", "Presión",
"Velocidad"])
media_temp = df["Temperatura"].mean()
desviacion_temp = df["Temperatura"].std()

# Identificar temperaturas fuera del rango normal
anomalos = df[df["Temperatura"] > media_temp + 2 * desviacion_temp]
print("Eventos anómalos detectados:")
print(anomalos)
```

Explicación:

- Calcula la media y desviación estándar de la temperatura.
- Detecta valores **anormales** que superen dos desviaciones estándar.

9.4 Integración con Sistemas de Notificación

Podemos generar alertas cuando se detecte un fallo en sensores o en el PLC.

9.4.1 Enviar Notificaciones a Telegram

Podemos usar Telegram para enviar alertas en caso de falla.

1. **Crear un bot en Telegram** usando [@BotFather](#).
2. **Obtener el token del bot** y el ID del chat.
3. **Instalar requests** en Python:
4. `pip install requests`
5. **Código para Enviar una Alerta a Telegram:**

```
import requests

TOKEN = "TU_BOT_TOKEN"
CHAT_ID = "TU_CHAT_ID"

def enviar_alerta_telegram(mensaje):
    url = f"https://api.telegram.org/bot{TOKEN}/sendMessage"
    params = {"chat_id": CHAT_ID, "text": mensaje}
    requests.get(url, params=params)

temp = 55.0 # Simulación de alta temperatura
```



```
if temp > 50.0:  
    enviar_alerta_telegram(f";ALERTA! Temperatura alta: {temp}°C")
```

Explicación:

- Envía un mensaje a Telegram si la temperatura supera 50°C.

9.5 Análisis de Tendencias y Reportes

Podemos generar **gráficos de tendencias** con los datos registrados.

9.5.1 Graficar la Temperatura con Matplotlib

```
import matplotlib.pyplot as plt  
  
df = pd.read_csv("datos_plc.csv", names=["Tiempo", "Temperatura", "Presión",  
"Velocidad"])  
df["Tiempo"] = pd.to_datetime(df["Tiempo"])  
  
plt.figure(figsize=(10,5))  
plt.plot(df["Tiempo"], df["Temperatura"], label="Temperatura")  
plt.xlabel("Tiempo")  
plt.ylabel("°C")  
plt.legend()  
plt.title("Tendencia de Temperatura en el PLC")  
plt.xticks(rotation=45)  
plt.show()
```

Explicación:

- Genera un gráfico de temperatura en el tiempo.
- Se pueden visualizar tendencias de fallos o anomalías.

9.6 Conclusión

- ✓ **Optimizamos la comunicación** con el PLC para mejorar el rendimiento.
- ✓ **Detectamos fallos** en la conexión y en los sensores.
- ✓ **Implementamos mantenimiento predictivo** con datos históricos.
- ✓ **Enviamos alertas a Telegram** en caso de anomalías.
- ✓ **Generamos gráficos** de tendencias para análisis industrial.

En el **próximo capítulo**, desarrollaremos un **proyecto final** que integre todo lo aprendido.



Capítulo 10: Proyecto Final - Sistema de Monitoreo y Control Completo

En este capítulo, desarrollaremos un **sistema de monitoreo y control completo** para un **PLC Siemens** utilizando **Python, Flask, MQTT, Telegram y una base de datos**. Integraremos todo lo aprendido en los capítulos anteriores para crear una solución funcional.

10.1 Objetivo del Proyecto

- ✓ Monitorear temperatura, presión y velocidad en **tiempo real**.
- ✓ **Activar o desactivar** una salida del PLC desde una interfaz web.
- ✓ **Registrar datos** en una base de datos **MySQL**.
- ✓ **Generar alertas** cuando la temperatura sea anómala.
- ✓ **Visualizar tendencias** con gráficos dinámicos.

10.2 Componentes del Proyecto

- ✓ PLC Siemens S7-1200/S7-1500 (o PLCSIM).
- ✓ Python (snap7, Flask, MySQL, MQTT, Telegram, Matplotlib).
- ✓ Base de datos MySQL para almacenar registros.
- ✓ Interfaz web con Flask y JavaScript para el control remoto.

10.3 Creación del Servidor Flask

Creamos un **servidor web** que se conecta al PLC, gestiona datos y proporciona una API para la interfaz web.

10.3.1 Código del Servidor Flask

```
from flask import Flask, jsonify, render_template
import snap7
import time
import mysql.connector
import paho.mqtt.client as mqtt
import requests
import struct
```



```
app = Flask(__name__)

# Conectar con el PLC
plc = snap7.client.Client()
plc.connect('192.168.1.100', 0, 1)

# Conectar con MySQL
conexion = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="plc_data"
)
cursor = conexion.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS registros (id INT AUTO_INCREMENT
PRIMARY KEY, temperatura FLOAT, presion FLOAT, velocidad FLOAT, tiempo
TIMESTAMP DEFAULT CURRENT_TIMESTAMP)")

# Configurar MQTT
broker = "test.mosquitto.org"
cliente_mqtt = mqtt.Client("PLC_Siemens")
cliente_mqtt.connect(broker)

# Configurar Telegram
TOKEN_TELEGRAM = "TU_BOT_TOKEN"
CHAT_ID = "TU_CHAT_ID"

def enviar_alerta_telegram(mensaje):
    url = f"https://api.telegram.org/bot{TOKEN_TELEGRAM}/sendMessage"
    params = {"chat_id": CHAT_ID, "text": mensaje}
    requests.get(url, params=params)

def leer_DB_real(plc, db_number, start_byte):
    data = plc.read_area(snap7.types.Areas.DB, db_number, start_byte, 4)
    return struct.unpack('>f', data)[0]

@app.route('/')
def index():
    return render_template("index.html")

@app.route('/estado', methods=['GET'])
def obtener_estado():
    temp = leer_DB_real(plc, 1, 0)
    presion = leer_DB_real(plc, 1, 4)
    velocidad = leer_DB_real(plc, 1, 8)

    # Guardar en la base de datos
    cursor.execute("INSERT INTO registros (temperatura, presion, velocidad)
VALUES (%s, %s, %s)", (temp, presion, velocidad))
    conexion.commit()

    # Publicar en MQTT
    cliente_mqtt.publish("plc/temperatura", temp)

    # Enviar alerta a Telegram si temperatura es alta
    if temp > 50.0:
        enviar_alerta_telegram(f";ALERTA! Temperatura alta: {temp}°C")
```



```

        return jsonify({"temperatura": temp, "presion": presion, "velocidad":
        velocidad})

@app.route('/activar_salida', methods=['GET'])
def activar_salida():
    escribir_salida(plc, 0, 1, 1)
    return jsonify({"mensaje": "Salida activada"})

@app.route('/desactivar_salida', methods=['GET'])
def desactivar_salida():
    escribir_salida(plc, 0, 1, 0)
    return jsonify({"mensaje": "Salida desactivada"})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Explicación: ✓ Se conecta al **PLC** para leer y controlar datos.

✓ Guarda los datos en **MySQL** cada vez que se actualizan.

✓ Publica los valores en **MQTT** para IoT.

✓ Envía **alertas a Telegram** si la temperatura es crítica.

✓ Proporciona una API REST para la **interfaz web**.

10.4 Creación de la Interfaz Web

La interfaz permite visualizar datos y controlar salidas del PLC.

10.4.1 Código HTML y JavaScript

Guarda este archivo como `templates/index.html`:

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Monitor PLC Siemens</title>
    <script>
        function actualizarEstado() {
            fetch('/estado')
            .then(response => response.json())
            .then(data => {
                document.getElementById("temperatura").innerText =
"Temperatura: " + data.temperatura + "°C";
                document.getElementById("presion").innerText = "Presión: " +
data.presion + " Bar";
                document.getElementById("velocidad").innerText = "Velocidad:
" + data.velocidad + " m/s";
            });
        }
    }

```



```

function activarSalida() {
    fetch('/activar_salida')
    .then(response => alert("Salida activada"));
}

function desactivarSalida() {
    fetch('/desactivar_salida')
    .then(response => alert("Salida desactivada"));
}

setInterval(actualizarEstado, 5000);
</script>
</head>
<body>
    <h1>Monitor PLC Siemens</h1>
    <p id="temperatura">Temperatura: --°C</p>
    <p id="presion">Presión: -- Bar</p>
    <p id="velocidad">Velocidad: -- m/s</p>
    <button onclick="activarSalida()">Activar Salida</button>
    <button onclick="desactivarSalida()">Desactivar Salida</button>
</body>
</html>

```

Explicación: ✓ Usa **JavaScript** para actualizar los datos cada 5 segundos.

✓ Permite **activar y desactivar** una salida del PLC.

✓ Se conecta a la API Flask para obtener datos en tiempo real.

10.5 Visualización de Datos y Análisis

Podemos graficar los datos de la base de datos con **Matplotlib**.

10.5.1 Gráfico de Tendencia de Temperatura

```

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_sql("SELECT * FROM registros", conexion)
df["tiempo"] = pd.to_datetime(df["tiempo"])

plt.figure(figsize=(10,5))
plt.plot(df["tiempo"], df["temperatura"], label="Temperatura")
plt.xlabel("Tiempo")
plt.ylabel("°C")
plt.title("Tendencia de Temperatura en el PLC")
plt.xticks(rotation=45)
plt.legend()
plt.show()

```



Explicación:

- Obtiene los datos desde **MySQL** y los grafica.
- Permite analizar el comportamiento de la temperatura.

10.6 Pruebas y Validación del Sistema

Para probar el sistema: ✓ **Ejecutar el servidor Flask** con `python servidor_plc.py`.

✓ **Abrir el navegador** en `http://localhost:5000`.

✓ **Modificar valores en el PLC** y verificar su actualización.

✓ **Activar/desactivar salidas** desde la interfaz web.

✓ **Verificar alertas en Telegram** cuando la temperatura sea alta.

10.7 Conclusión

✓ Desarrollamos un **sistema completo de monitoreo y control** para un PLC Siemens.

✓ **Almacenamos datos en MySQL** y los analizamos con gráficos.

✓ **Enviamos notificaciones a Telegram** cuando hay problemas.

✓ **Publicamos datos en MQTT** para integración IoT.

✓ **Creamos una interfaz web dinámica** para controlar el PLC desde cualquier navegador.

Este **proyecto final** integra todo lo aprendido en el libro y puede expandirse con más funcionalidades, como dashboards en tiempo real, control remoto desde apps móviles y predicción de fallos con IA.



Anexo: Glosario de Términos

A continuación, se presenta un glosario con los términos más utilizados en la programación de **PLCs Siemens con Python**, facilitando su comprensión.

A

- **Automatización Industrial:** Uso de tecnología para controlar procesos industriales sin intervención humana.
- **API (Application Programming Interface):** Conjunto de funciones que permite la comunicación entre sistemas o aplicaciones.

B

- **Base de Datos (DB):** Sistema para almacenar y organizar datos. En los PLCs, se usan bloques de datos (DB) para guardar valores internos.
- **Bit:** Unidad mínima de información en un PLC, puede ser **0 (apagado)** o **1 (encendido)**.
- **Broker (MQTT):** Servidor que gestiona la comunicación en el protocolo MQTT.

C

- **Ciclo de Escaneo (Scan Cycle):** Proceso en el que el PLC lee entradas, ejecuta el programa y actualiza las salidas en un bucle continuo.
- **CPU (Unidad Central de Procesamiento):** Componente principal del PLC que ejecuta el programa lógico.
- **CSV (Comma-Separated Values):** Formato de archivo para almacenar datos en texto, separado por comas.

D

- **Dato Analógico:** Valor variable (ejemplo: temperatura, presión).
- **Dato Digital:** Estado binario (0 o 1).
- **Direccionamiento de Memoria:** Forma en la que el PLC accede a entradas, salidas y registros internos.

E

- **Entrada (I):** Señal que el PLC recibe desde sensores o dispositivos externos.
- **Escritura de Datos:** Proceso de enviar valores desde Python al PLC.



F

- **Flask:** Framework ligero de Python para crear aplicaciones web.
- **Firmware:** Software embebido en el PLC que controla su funcionamiento.

H

- **HMI (Human Machine Interface):** Interfaz gráfica para que el operador interactúe con el PLC.

I

- **IP (Internet Protocol):** Dirección única asignada a dispositivos en una red, como los PLCs.

L

- **LAD (Ladder Diagram):** Lenguaje de programación basado en diagramas de escalera, usado en PLCs.
- **Latencia:** Retraso en la comunicación entre dispositivos.

M

- **MQTT (Message Queuing Telemetry Transport):** Protocolo de comunicación ligero usado en IoT.
- **Marca (M):** Memoria interna del PLC para almacenar valores temporales.

N

- **Node-RED:** Plataforma visual para la integración de sistemas industriales y IoT.

O

- **Operación Lógica:** Procesos como AND, OR, NOT utilizados en la programación de PLCs.

P

- **PLCSIM:** Simulador de PLCs de Siemens que permite probar programas sin hardware físico.
- **Protocolo de Comunicación:** Conjunto de reglas que permiten la transmisión de datos entre dispositivos (ejemplo: S7, MQTT, Modbus).



- **Python-Snap7**: Biblioteca de Python para comunicarse con PLCs Siemens.

Q

- **Salida (Q)**: Señal enviada por el PLC para activar actuadores como motores y válvulas.

R

- **RAM (Memoria de Acceso Aleatorio)**: Memoria temporal del PLC que se borra al apagarlo.
- **Real (REAL)**: Tipo de dato en el PLC que almacena valores decimales (flotantes).
- **Registro**: Entrada individual en una base de datos o bloque de datos del PLC.

S

- **SCADA (Supervisory Control and Data Acquisition)**: Sistema de supervisión y control de procesos industriales.
- **Snap7**: Biblioteca de código abierto que permite la comunicación con PLCs Siemens.
- **S7-1200 / S7-1500**: Modelos de PLCs de Siemens.
- **STRING**: Tipo de dato en el PLC que almacena texto.
- **SQL (Structured Query Language)**: Lenguaje de consulta usado en bases de datos.

T

- **TIA Portal (Totally Integrated Automation Portal)**: Entorno de desarrollo de Siemens para programar y configurar PLCs.
- **Telegram Bot**: Servicio de mensajería automatizado que puede enviar alertas desde Python.

U

- **UDP (User Datagram Protocol)**: Protocolo de red usado en la comunicación con PLCs.
- **Usuario Remoto**: Persona que accede al PLC desde otra ubicación mediante una interfaz web.

V

- **Variable de Proceso**: Parámetro medido en un sistema industrial, como temperatura o presión.



W

- **Watchdog Timer:** Temporizador que detecta si un sistema deja de responder.

Conclusión

Este **glosario** proporciona definiciones clave para entender mejor la **programación de PLCs Siemens con Python**, ayudando a recordar conceptos técnicos esenciales para proyectos de automatización.

